LAPORAN PRAKTIKUM
PENGOLAHAN CITRA DIGITAL

PENGENALAN GIT DAN GITHUB

Catatan Pembelajaran Sistem Kontrol Versi

Disusun oleh:

Nama	**:**	**M. Awaludin Ramadan**
NIM	**:**	**24260027**
Kelas	**:**	**Teknik Informatika 5.B**
Mata Kuliah	**:**	**Pengolahan Citra Digital**
Program Studi	**:**	**Teknik Informatika**

PROGRAM STUDI TEKNIK INFORMATIKA
UNIVERSITAS NAHDLATUL ULAMA INDONESIA BOGOR

TAHUN 2026

DAFTAR ISI

BAB I PENDAHULUAN 1
1.1 Latar Belakang 1
1.2 Rumusan Masalah 2
1.3 Tujuan Praktik 2
1.4 Manfaat Praktik 3
BAB II LANDASAN TEORI 4
2.1 Pengertian Git 4
2.2 Pengertian GitHub 4
2.3 Sejarah Perkembangan Git dan GitHub 5
2.4 Fungsi dan Manfaat Git 6
2.5 Fungsi dan Manfaat GitHub 6
2.6 Pengertian Version Control System 7
2.7 Pengertian Repository 8
2.8 Perbedaan Git dan GitHub 8
2.9 Istilah-Istilah Penting dalam GitHub 9
2.10 Perintah Dasar Git 10
BAB III MATERI DAN PEMBAHASAN 13
3.1 Mengenal Tampilan dan Fitur GitHub 13
3.2 Cara Membuat Akun GitHub 13
3.3 Cara Membuat Repository 14
3.4 Cara Mengunggah File ke GitHub 15
3.5 Cara Membuat dan Mengedit README.md 15
3.6 Cara Menggunakan Git di Komputer 17
3.7 Cara Menghubungkan Git dengan GitHub 18
3.8 Cara Melakukan Git Add, Commit, dan Push 19
3.9 Cara Melakukan Git Pull dan Clone 20
3.10 Mengenal Branch dan Merge 20
3.11 Mengenal Fork dan Pull Request 22
3.12 Keamanan dan Pengelolaan Repository 23
BAB IV HASIL PEMBELAJARAN 25
4.1 Pemahaman Mengenai Git dan GitHub 25
4.2 Pemahaman Fungsi Repository 25
4.3 Pemahaman Perintah Dasar Git 25
4.4 Pemahaman Proses Upload File ke GitHub 26
4.5 Manfaat GitHub dalam Dunia Pemrograman 26
4.6 Penerapan GitHub dalam Kegiatan Perkuliahan 27
BAB V PENUTUP 28
5.1 Kesimpulan 28
5.2 Saran 28
DAFTAR PUSTAKA 29

BAB I
PENDAHULUAN

1.1 Latar Belakang

Perkembangan teknologi informasi membuat pekerjaan di bidang pemrograman, pengolahan data, dan pengolahan citra tidak lagi dikerjakan sendirian dengan satu berkas saja. Sebuah pekerjaan biasanya terdiri dari banyak berkas, misalnya kode program, dokumen catatan, data pendukung, dan laporan. Berkas-berkas tersebut juga terus berubah dari waktu ke waktu. Tanpa pengelolaan yang baik, perubahan ini mudah menimbulkan kebingungan. Contoh yang sering dialami mahasiswa adalah munculnya berkas dengan nama seperti `tugas_final`, `tugas_final_revisi`, dan `tugas_final_revisi_baru`, sehingga sulit menentukan berkas mana yang benar-benar terbaru.

Masalah tersebut dapat diatasi dengan *version control system* atau sistem kontrol versi. Salah satu sistem kontrol versi yang paling banyak digunakan saat ini adalah **Git**. Git mencatat setiap perubahan pada berkas sehingga pengguna dapat melihat riwayat perubahan, membandingkan versi, dan kembali ke versi sebelumnya apabila terjadi kesalahan. Agar riwayat dan berkas tersebut dapat disimpan di internet serta dikerjakan bersama orang lain, Git umumnya dipadukan dengan **GitHub**, yaitu layanan *hosting* repositori berbasis web yang menggunakan Git sebagai dasarnya.

Bagi mahasiswa Teknik Informatika, kemampuan menggunakan Git dan GitHub merupakan keterampilan dasar yang penting. Dalam mata kuliah Pengolahan Citra Digital, misalnya, mahasiswa akan banyak berhubungan dengan berkas kode, hasil percobaan, dan catatan yang perlu dirapikan dan diberi versi. Karena itu, sebelum mempelajari topik pengolahan citra secara mendalam, mahasiswa perlu memahami alat bantu pengelolaan berkas ini terlebih dahulu.

Laporan ini disusun sebagai catatan pembelajaran mengenai pengenalan Git dan GitHub secara umum. Laporan ini bukan laporan hasil proyek pemrograman atau pengolahan citra digital, melainkan rangkuman pemahaman penulis mengenai konsep, istilah, perintah dasar, dan alur kerja Git dan GitHub yang diperoleh selama kegiatan praktikum.

1.2 Rumusan Masalah

Berdasarkan latar belakang di atas, rumusan masalah dalam laporan ini adalah sebagai berikut.

1. Apa yang dimaksud dengan Git dan GitHub, serta bagaimana sejarah perkembangannya?
2. Apa fungsi dan manfaat Git dan GitHub dalam pengelolaan berkas dan kerja sama tim?
3. Apa perbedaan antara Git dan GitHub, serta apa saja istilah penting yang perlu dipahami?
4. Bagaimana langkah dasar menggunakan GitHub, mulai dari membuat akun, membuat repositori, hingga mengunggah berkas?
5. Bagaimana cara menggunakan perintah dasar Git di komputer dan menghubungkannya dengan GitHub?
6. Bagaimana konsep *branch*, *merge*, *fork*, dan *pull request* serta cara menjaga keamanan repositori?

1.3 Tujuan Praktik

Tujuan kegiatan praktikum dan penyusunan laporan ini adalah sebagai berikut.

1. Menjelaskan pengertian, sejarah, fungsi, dan manfaat Git serta GitHub.
2. Menjelaskan konsep sistem kontrol versi dan repositori.
3. Membedakan Git dan GitHub serta memahami istilah-istilah penting yang digunakan.
4. Memahami langkah membuat akun GitHub, membuat repositori, mengunggah berkas, dan menulis berkas README.md.
5. Memahami penggunaan perintah dasar Git seperti `git init`, `git add`, `git commit`, `git push`, `git pull`, dan `git clone`.
6. Memahami konsep percabangan (*branch*), penggabungan (*merge*), *fork*, *pull request*, serta dasar keamanan dan pengelolaan repositori.

1.4 Manfaat Praktik

Manfaat yang diharapkan dari kegiatan praktikum dan laporan ini adalah sebagai berikut.

- **Bagi penulis:** menambah pemahaman dan keterampilan dasar dalam mengelola berkas dan versi, sekaligus menjadi bahan belajar ulang apabila suatu saat lupa terhadap materi.
- **Bagi mahasiswa lain:** menjadi bahan bacaan awal yang ringkas dan mudah dipahami bagi pembaca yang baru mengenal Git dan GitHub.
- **Bagi mata kuliah dan program studi:** mendukung kesiapan mahasiswa dalam mengerjakan tugas, proyek kelompok, dan penelitian yang membutuhkan pengelolaan kode dan dokumen secara rapi serta terdokumentasi.

BAB II
LANDASAN TEORI

2.1 Pengertian Git

Git adalah perangkat lunak sistem kontrol versi terdistribusi (*distributed version control system*) yang bersifat gratis dan *open source*. Git dipasang dan dijalankan di komputer pengguna, kemudian bertugas mencatat perubahan yang terjadi pada sekumpulan berkas dari waktu ke waktu. Menurut dokumentasi resminya, Git dirancang untuk menangani proyek dari skala kecil hingga sangat besar dengan cepat dan efisien (Git, n.d.-a).

Cara kerja Git dapat dibayangkan seperti fitur *save point* pada permainan video. Setiap kali pengguna merasa pekerjaannya sudah mencapai kondisi yang layak disimpan, ia membuat satu titik simpan yang disebut *commit*. Apabila di kemudian hari pekerjaan terasa salah arah, pengguna dapat kembali ke titik simpan sebelumnya. Berbeda dengan sekadar menyalin folder, Git menyimpan riwayat secara terstruktur sehingga setiap titik simpan memiliki identitas, penulis, waktu, dan catatan perubahan.

Chacon dan Straub (2014) menjelaskan bahwa Git memandang datanya sebagai rangkaian potret keadaan (*snapshot*) dari berkas pada suatu waktu, bukan sekadar daftar selisih perubahan. Selain itu, sebagian besar operasi Git dilakukan di komputer lokal sehingga cepat dan tetap dapat digunakan tanpa koneksi internet. Git juga menjaga keutuhan data dengan *checksum*, sehingga perubahan pada isi berkas tanpa sepengetahuan Git dapat terdeteksi.

2.2 Pengertian GitHub

GitHub adalah platform berbasis web yang menyediakan layanan penyimpanan (*hosting*) repositori Git di internet serta berbagai fitur untuk kolaborasi. Dokumentasi GitHub menyebutkan bahwa GitHub digunakan untuk menyimpan, melacak, dan berkolaborasi dalam proyek perangkat lunak, dengan Git sebagai teknologi yang berjalan di dalamnya (GitHub Docs, n.d.-a).

Dengan GitHub, repositori yang semula hanya ada di satu komputer dapat disimpan di *cloud*, diakses dari perangkat lain, dan dikerjakan bersama orang lain. GitHub juga menambahkan fitur yang tidak ada pada Git murni, antara lain *issues*

untuk mencatat tugas atau masalah, *pull request* untuk mengusulkan perubahan, *GitHub Actions* untuk otomatisasi, *GitHub Pages* untuk menerbitkan situs statis, serta halaman profil yang dapat berfungsi sebagai portofolio. Selain GitHub, terdapat layanan serupa seperti GitLab dan Bitbucket yang juga menggunakan Git.

2.3 Sejarah Perkembangan Git dan GitHub

a. Sejarah Git

Git lahir dari kebutuhan proyek *kernel* Linux. Pada awalnya para pengembang Linux menggunakan sistem kontrol versi terdistribusi komersial bernama BitKeeper. Pada tahun 2005 hubungan antara komunitas Linux dengan perusahaan pemilik BitKeeper memburuk dan hak penggunaan gratisnya dicabut. Hal ini mendorong Linus Torvalds, pencipta Linux, untuk membuat sistem kontrol versi sendiri (Chacon & Straub, 2014; Git, n.d.-b).

Beberapa tujuan rancangan Git pada saat itu adalah kecepatan, rancangan yang sederhana, dukungan kuat untuk pengembangan non-linear dengan ribuan cabang paralel, sifat terdistribusi sepenuhnya, serta kemampuan menangani proyek besar seperti kernel Linux secara efisien. Sejak dirilis, Git terus dikembangkan oleh komunitas dan kini menjadi salah satu sistem kontrol versi yang paling banyak digunakan di dunia.

b. Sejarah GitHub

GitHub didirikan pada tahun 2008 oleh Tom Preston-Werner, Chris Wanstrath, P. J. Hyett, dan Scott Chacon, dengan tujuan mempermudah pengembang berbagi dan berkolaborasi pada repositori Git. Seiring waktu GitHub berkembang menjadi pusat komunitas pengembang perangkat lunak, termasuk proyek *open source*. Pada tahun 2018 Microsoft mengumumkan akuisisi GitHub dan proses tersebut diselesaikan pada bulan Oktober 2018 (Microsoft, 2018a; Microsoft, 2018b). Setelah akuisisi, GitHub tetap beroperasi sebagai platform yang dapat digunakan oleh siapa saja.

Beberapa perubahan penting yang dapat dicatat adalah penggunaan nama cabang bawaan `main` pada repositori baru di GitHub sebagai pengganti `master`, serta dihentikannya autentikasi menggunakan kata sandi biasa untuk operasi Git melalui HTTPS. Sebagai gantinya, pengguna memakai *personal access token* atau kunci SSH (GitHub Docs, n.d.-b).

2.4 Fungsi dan Manfaat Git

Berdasarkan materi yang dipelajari, Git memiliki fungsi dan manfaat berikut.

- **Mencatat riwayat perubahan.** Setiap perubahan yang di-*commit* tersimpan lengkap dengan pesan, penulis, dan waktunya sehingga mudah ditelusuri.
- **Kembali ke versi sebelumnya.** Jika terjadi kesalahan, pengguna dapat memulihkan berkas ke kondisi yang lalu tanpa harus mengulang pekerjaan dari awal.
- **Bekerja pada beberapa fitur secara paralel.** Fitur *branch* memungkinkan pengguna mencoba ide baru di cabang terpisah tanpa mengganggu versi utama.
- **Mendukung kerja tim.** Beberapa orang dapat mengerjakan proyek yang sama, kemudian hasil kerjanya digabungkan dan konflik dapat diselesaikan secara terkendali.
- **Bekerja secara lokal.** Sebagian besar perintah tidak membutuhkan internet sehingga cepat dan tetap dapat dipakai saat luring.
- **Menjadi cadangan.** Karena setiap salinan repositori berisi riwayat lengkap, repositori yang rusak atau hilang dapat dipulihkan dari salinan lain.

2.5 Fungsi dan Manfaat GitHub

GitHub memberikan fungsi dan manfaat tambahan di atas kemampuan Git, yaitu sebagai berikut.

- **Penyimpanan *cloud*.** Repositori tersimpan di internet sehingga dapat diakses dari mana saja dan berfungsi sebagai cadangan.
- **Kolaborasi.** Fitur *issues*, *pull request*, *code review*, dan *discussions* membantu tim berkomunikasi dan mengevaluasi perubahan.
- **Dokumentasi proyek.** Berkas README.md otomatis ditampilkan di halaman utama repositori sehingga proyek mudah dipahami pengunjung.
- **Otomatisasi.** *GitHub Actions* dapat menjalankan proses otomatis, seperti pengujian kode, ketika terjadi suatu kejadian pada repositori.
- **Publikasi.** *GitHub Pages* dapat dipakai untuk menerbitkan situs statis, misalnya dokumentasi atau portofolio.

- **Portofolio dan komunitas.** Profil GitHub memperlihatkan repositori dan aktivitas pengguna, serta memudahkan berkontribusi pada proyek *open source*.
- **Pengendalian akses.** Repositori dapat dibuat publik atau privat, dan hak akses kolaborator dapat diatur.

2.6 Pengertian Version Control System

Version Control System (VCS) atau sistem kontrol versi adalah sistem yang mencatat perubahan pada berkas dari waktu ke waktu sehingga versi tertentu dapat dipanggil kembali di kemudian hari (Chacon & Straub, 2014; Git, n.d.-c). VCS tidak hanya dipakai untuk kode program, tetapi dapat dipakai untuk hampir semua jenis berkas, termasuk dokumen dan gambar. Dengan VCS, pengguna dapat mengetahui apa yang berubah, kapan perubahan terjadi, dan siapa yang melakukannya.

Menurut Chacon dan Straub (2014), sistem kontrol versi berkembang dalam tiga jenis utama yang dirangkum pada Tabel 2.1.

Tabel 2.1 Jenis-jenis Sistem Kontrol Versi

Jenis	Cara Kerja	Kelebihan dan Kekurangan	Contoh
Lokal (*Local VCS*)	Perubahan dicatat dalam basis data sederhana di satu komputer.	Sederhana, tetapi sulit untuk kerja tim dan berisiko hilang jika komputer rusak.	RCS
Terpusat (*Centralized VCS*)	Seluruh versi disimpan di satu server pusat; pengguna mengambil berkas dari server.	Mudah dikelola dan kolaboratif, tetapi bergantung pada server; jika server mati atau rusak, riwayat dapat hilang.	Subversion (SVN), CVS
Terdistribusi (*Distributed VCS*)	Setiap pengguna menyalin repositori beserta seluruh riwayatnya.	Dapat bekerja luring dan tahan terhadap kerusakan server karena ada banyak salinan lengkap.	Git, Mercurial

Contoh sederhananya adalah tugas kelompok. Pada sistem lokal, riwayat hanya ada di komputer satu orang. Pada sistem terpusat, semua orang bergantung pada satu server. Pada sistem terdistribusi seperti Git, setiap anggota memiliki

salinan lengkap, sehingga pekerjaan tidak hilang walaupun satu komputer atau server bermasalah.

2.7 Pengertian Repository

Repository atau repositori adalah tempat penyimpanan sebuah proyek beserta seluruh riwayat perubahannya. Dokumentasi GitHub menjelaskan bahwa repositori berisi berkas-berkas proyek dan menyimpan riwayat revisi setiap berkas (GitHub Docs, n.d.-c). Repositori dapat dianalogikan sebagai sebuah folder proyek yang memiliki ingatan lengkap tentang semua perubahan yang pernah terjadi di dalamnya.

Secara teknis, ketika seseorang menjalankan `git init` di sebuah folder, Git membuat sub-folder tersembunyi bernama `.git`. Folder inilah yang menyimpan basis data riwayat, konfigurasi, dan data lain milik repositori. Menghapus folder `.git` berarti menghapus riwayat, sehingga folder tersebut tidak lagi menjadi repositori Git.

Repositori dikenal dalam dua bentuk berikut.

- **Repositori lokal**, yaitu repositori yang berada di komputer pengguna.
- **Repositori jarak jauh (*remote*)**, yaitu repositori yang berada di server, misalnya di GitHub.

Di GitHub, repositori dapat berstatus **publik** sehingga dapat dilihat siapa saja, atau **privat** sehingga hanya dapat diakses oleh pemilik dan orang yang diberi izin.

2.8 Perbedaan Git dan GitHub

Git dan GitHub sering dianggap sama karena selalu disebut bersamaan. Padahal keduanya berbeda. Git adalah alat atau perangkat lunak, sedangkan GitHub adalah layanan di internet yang memanfaatkan alat tersebut. Perbedaannya dirangkum pada Tabel 2.2.

Tabel 2.2 Perbedaan Git dan GitHub

Aspek	Git	GitHub
Jenis	Perangkat lunak sistem kontrol versi	Layanan *hosting* repositori berbasis web
Lokasi kerja	Terpasang dan berjalan di komputer pengguna	Berjalan di *cloud* (internet)

Aspek	Git	GitHub
Pembuat	Linus Torvalds (2005)	Didirikan 2008; kini dimiliki Microsoft
Kebutuhan internet	Tidak wajib untuk sebagian besar perintah	Wajib untuk mengakses dan menyinkronkan
Antarmuka	Terutama *command line*	Situs web, aplikasi desktop, dan aplikasi seluler
Fungsi utama	Mencatat dan mengelola versi berkas	Menyimpan repositori online dan mendukung kolaborasi
Fitur khas	*commit*, *branch*, *merge*, *log*	*issues*, *pull request*, *Actions*, *Pages*, profil
Alternatif	Mercurial, Subversion	GitLab, Bitbucket

Sebagai perumpamaan, Git ibarat aplikasi pengolah foto yang terpasang di komputer, sedangkan GitHub ibarat media sosial tempat hasil karya tersebut diunggah, dipamerkan, dan dikomentari bersama. Seseorang tetap dapat memakai Git tanpa GitHub, tetapi GitHub tidak dapat berfungsi tanpa Git.

2.9 Istilah-Istilah Penting dalam GitHub

Dalam mempelajari Git dan GitHub terdapat banyak istilah dalam bahasa Inggris. Istilah yang dipelajari dirangkum pada Tabel 2.3 beserta artinya menurut pemahaman penulis.

Tabel 2.3 Istilah Penting dalam Git dan GitHub

Istilah	Arti Singkat
Repository (repo)	Tempat penyimpanan proyek beserta riwayat perubahannya.
Commit	Titik simpan yang berisi potret perubahan beserta pesan penjelasnya.
Staging area	Area antara tempat berkas yang dipilih untuk dimasukkan ke *commit* berikutnya.
Branch	Cabang atau jalur pengembangan terpisah dari jalur utama.
Main	Nama cabang utama yang umum dipakai pada repositori baru.
Merge	Penggabungan perubahan dari satu cabang ke cabang lain.
Remote	Repositori yang berada di server lain, misalnya di GitHub.

Istilah	Arti Singkat
Origin	Nama bawaan untuk *remote* utama.
Clone	Menyalin repositori dari GitHub ke komputer lengkap dengan riwayatnya.
Push	Mengirim *commit* dari repositori lokal ke *remote*.
Pull	Mengambil dan menggabungkan perubahan terbaru dari *remote* ke repositori lokal.
Fetch	Mengambil data terbaru dari *remote* tanpa langsung menggabungkannya.
Fork	Salinan repositori milik orang lain ke akun sendiri.
Pull request (PR)	Permintaan agar perubahan di satu cabang atau *fork* ditinjau dan digabungkan.
Issue	Catatan tugas, pertanyaan, atau laporan masalah pada repositori.
HEAD	Penunjuk posisi *commit* atau cabang yang sedang aktif.
Conflict	Pertentangan yang muncul ketika dua perubahan mengubah bagian yang sama.
README.md	Berkas penjelasan proyek yang tampil di halaman depan repositori.
`.gitignore`	Berkas berisi daftar berkas atau folder yang tidak perlu dilacak Git.
Collaborator	Orang lain yang diberi izin ikut mengelola repositori.
Tag dan *Release*	Penanda versi tertentu, misalnya v1.0, dan paket rilisnya di GitHub.
Star dan *Watch*	Menandai repositori favorit dan mengikuti aktivitasnya.
GitHub Actions	Fitur otomatisasi alur kerja pada repositori.
License	Lisensi yang mengatur hak penggunaan dan distribusi kode.

2.10 Perintah Dasar Git

Git digunakan melalui antarmuka baris perintah (*command line*), misalnya Git Bash di Windows atau terminal di macOS dan Linux. Sebelum membahas perintahnya, perlu dipahami bahwa berkas di Git berpindah melalui tiga area, yaitu *working directory* (folder kerja tempat berkas diubah), *staging area* (tempat berkas dipilih untuk disimpan), dan *repository* (tempat riwayat disimpan). Setelah itu berkas dapat dikirim ke *remote repository*, misalnya GitHub. Alurnya digambarkan sebagai berikut.

```
Working Directory  (berkas yang sedang diedit)
      |  git add
      v
Staging Area       (berkas yang siap di-commit)
      |  git commit
      v
Local Repository   (riwayat tersimpan di komputer)
      |  git push              ^  git pull / git fetch
      v                        |
Remote Repository  (misalnya di GitHub)
```

Perintah dasar yang dipelajari beserta fungsi dan contohnya dirangkum pada Tabel 2.4.

Tabel 2.4 Perintah Dasar Git

Perintah	Fungsi	Contoh
`git --version`	Menampilkan versi Git yang terpasang.	`git --version`
`git config`	Mengatur konfigurasi, misalnya nama dan email pengguna.	`git config --global user.name "Nama"`
`git init`	Membuat repositori Git baru di folder saat ini.	`git init`
`git clone`	Menyalin repositori dari *remote* ke komputer.	`git clone https://github.com/user/repo.git`
`git status`	Menampilkan status berkas (baru, berubah, atau sudah di-*stage*).	`git status`
`git add`	Memasukkan berkas ke *staging area*.	`git add catatan.txt` `git add .`
`git commit`	Menyimpan isi *staging area* sebagai titik simpan baru.	`git commit -m "Tambah catatan"`
`git log`	Menampilkan riwayat *commit*.	`git log --oneline`
`git diff`	Menampilkan perbedaan isi berkas.	`git diff`
`git branch`	Melihat atau membuat cabang.	`git branch fitur-baru`
`git switch`	Berpindah ke cabang lain.	`git switch fitur-baru`

Perintah	Fungsi	Contoh
`git merge`	Menggabungkan cabang ke cabang aktif.	`git merge fitur-baru`
`git remote`	Mengelola alamat *remote* repositori.	`git remote add origin URL`
`git push`	Mengirim *commit* lokal ke *remote*.	`git push origin main`
`git pull`	Mengambil dan menggabungkan perubahan dari *remote*.	`git pull origin main`
`git fetch`	Mengambil data terbaru dari *remote* tanpa menggabungkan.	`git fetch origin`
`git restore`	Mengembalikan berkas ke keadaan sebelumnya atau mengeluarkannya dari *staging*.	`git restore catatan.txt`
`git revert`	Membuat *commit* baru yang membatalkan *commit* tertentu.	`git revert HEAD`

BAB III
MATERI DAN PEMBAHASAN

3.1 Mengenal Tampilan dan Fitur GitHub

Setelah masuk ke akun, pengguna akan melihat halaman utama (*dashboard*) yang memuat daftar repositori, aktivitas terbaru, dan rekomendasi. Tampilan GitHub dapat berubah dari waktu ke waktu, tetapi komponen utamanya relatif sama. Komponen yang dikenali pada bagian atas halaman adalah kolom pencarian, tombol + untuk membuat repositori baru, ikon notifikasi, serta foto profil yang membuka menu berisi *Your profile*, *Your repositories*, *Settings*, dan *Sign out*.

Ketika membuka sebuah repositori, bagian terpenting adalah deretan tab di bagian atas. Fungsi masing-masing tab dirangkum pada Tabel 3.1.

Tabel 3.1 Tab Utama pada Halaman Repositori

Tab	Fungsi
Code	Menampilkan daftar berkas dan folder, pilihan cabang, tombol *Code* untuk *clone*, serta isi README.md.
Issues	Mencatat tugas, ide, pertanyaan, atau laporan masalah.
Pull requests	Mengelola usulan perubahan yang menunggu ditinjau dan digabungkan.
Actions	Mengelola alur kerja otomatis (CI/CD).
Projects	Papan perencanaan, mirip papan kanban, untuk mengatur pekerjaan.
Wiki	Halaman dokumentasi tambahan untuk proyek.
Security	Informasi keamanan, misalnya peringatan kerentanan dan pemindaian rahasia.
Insights	Statistik repositori, seperti kontributor dan aktivitas *commit*.
Settings	Pengaturan repositori: nama, visibilitas, kolaborator, perlindungan cabang, dan lainnya.

Selain itu terdapat tombol *Star* untuk menandai repositori favorit, *Watch* untuk mengikuti aktivitas, dan *Fork* untuk menyalin repositori ke akun sendiri. Di bagian samping halaman *Code* biasanya terlihat deskripsi singkat, bahasa pemrograman yang digunakan, dan informasi rilis.

3.2 Cara Membuat Akun GitHub

Langkah membuat akun GitHub yang dipelajari adalah sebagai berikut.

1. Buka peramban dan kunjungi situs `github.com`.
2. Klik tombol **Sign up** di pojok kanan atas.
3. Isi alamat surel, kata sandi, dan nama pengguna (*username*). Gunakan surel yang aktif karena akan menerima kode verifikasi.
4. Selesaikan verifikasi keamanan (misalnya teka-teki gambar) lalu klik **Create account**.
5. Masukkan kode verifikasi yang dikirim ke surel.
6. Ikuti pertanyaan awal (personalisasi) atau lewati, lalu pilih paket **Free** yang sudah mencukupi untuk kebutuhan belajar.
7. Aktifkan autentikasi dua faktor pada menu *Settings* agar akun lebih aman.

Beberapa catatan penting: nama pengguna sebaiknya sopan, singkat, dan profesional karena akan tampil pada alamat profil (`github.com/namapengguna`) dan sering dilihat dosen, rekan, maupun calon pemberi kerja. Mahasiswa juga dapat memanfaatkan program *GitHub Education* untuk memperoleh akses ke fasilitas belajar tambahan dengan memverifikasi status pelajar.

3.3 Cara Membuat Repository

Langkah membuat repositori baru melalui situs GitHub adalah sebagai berikut (GitHub Docs, n.d.-d).

1. Klik tombol **+** di bagian atas halaman lalu pilih **New repository**, atau klik tombol **New** pada daftar repositori.
2. Isi **Repository name**, misalnya `catatan-git-github`. Gunakan huruf kecil dan tanda hubung tanpa spasi agar rapi.
3. Isi **Description** secara singkat sebagai penjelasan isi repositori.
4. Pilih visibilitas: **Public** (dapat dilihat semua orang) atau **Private** (hanya pemilik dan orang yang diizinkan).
5. Centang **Add a README file** agar repositori langsung memiliki berkas penjelasan.
6. Jika perlu, pilih templat **.gitignore** dan **License**.
7. Klik **Create repository**.

Setelah repositori dibuat, GitHub menampilkan halaman repositori yang berisi berkas awal. Alamat repositori berbentuk `https://github.com/namapengguna/catatan-git-github`. Alamat inilah yang nanti dipakai untuk menghubungkan repositori lokal dengan GitHub.

3.4 Cara Mengunggah File ke GitHub

Berkas dapat diunggah ke GitHub dengan dua cara, yaitu melalui peramban (tanpa Git di komputer) atau melalui perintah Git. Cara pertama cocok untuk pemula, sedangkan cara kedua dibahas pada subbab 3.8.

Langkah mengunggah berkas melalui peramban (GitHub Docs, n.d.-e) adalah sebagai berikut.

1. Buka halaman repositori pada tab **Code**.
2. Klik tombol **Add file**, lalu pilih **Upload files**.
3. Seret berkas dari komputer ke area unggah, atau klik **choose your files** untuk memilih berkas.
4. Pada bagian **Commit changes**, tulis pesan singkat yang menjelaskan unggahan, misalnya `Menambahkan catatan pertemuan 1`.
5. Pilih untuk melakukan *commit* langsung ke cabang utama atau membuat cabang baru bila ingin ditinjau lebih dahulu.
6. Klik **Commit changes**. Berkas kini tampil pada daftar berkas repositori.

Perlu diperhatikan bahwa unggahan melalui peramban memiliki batas ukuran berkas, dan Git/GitHub tidak cocok untuk menyimpan berkas yang sangat besar. Untuk berkas besar, GitHub menyediakan *Git Large File Storage* (Git LFS). Berkas yang bersifat rahasia, seperti kata sandi atau kunci API, tidak boleh diunggah, terutama ke repositori publik.

3.5 Cara Membuat dan Mengedit README.md

README.md adalah berkas penjelasan proyek. Berkas ini otomatis ditampilkan di halaman utama repositori sehingga menjadi bagian pertama yang dibaca pengunjung (GitHub Docs, n.d.-f). Ekstensi `.md` menunjukkan bahwa isinya ditulis dengan format **Markdown**, yaitu bahasa penulisan sederhana yang menggunakan simbol untuk mengatur tampilan teks (GitHub Docs, n.d.-g).

a. Membuat README.md

Jika belum dicentang saat membuat repositori, README.md dapat dibuat dengan menekan **Add file**, memilih **Create new file**, lalu menamai berkas `README.md`. Jika sudah ada, cukup membuka berkas tersebut.

b. Mengedit README.md

Pengeditan dapat dilakukan langsung di GitHub dengan langkah berikut.

1. Buka berkas README.md pada daftar berkas.
2. Klik ikon pensil (**Edit this file**).
3. Ubah isi pada kolom penyunting. Tab **Preview** dapat digunakan untuk melihat hasil tampilan.
4. Klik **Commit changes**, tulis pesan, lalu simpan.

c. Sintaks dasar Markdown

Tabel 3.2 Sintaks Dasar Markdown

Tujuan	Penulisan	Hasil
Judul besar	`# Judul`	Judul tingkat 1
Subjudul	`## Subjudul`	Judul tingkat 2
Teks tebal	`**tebal**`	**tebal**
Teks miring	`*miring*`	*miring*
Daftar poin	`- butir`	Daftar berpoin
Daftar bernomor	`1. langkah`	Daftar bernomor
Tautan	`[teks](https://alamat)`	Teks yang dapat diklik
Kode satu baris	teks diapit tanda *backtick*	Teks berfont kode

Contoh isi README.md sederhana:

```
# Catatan Belajar Git dan GitHub

Repositori ini berisi catatan belajar mata kuliah
Pengolahan Citra Digital.

## Isi Repositori
- `catatan.txt` : catatan pertemuan pertama
- `perintah-git.md` : rangkuman perintah dasar Git

## Penulis
M. Awaludin Ramadan - 24260027
```

3.6 Cara Menggunakan Git di Komputer

a. Memasang Git

Git dapat diunduh secara gratis dari situs resmi `git-scm.com/downloads` (Git, n.d.-d). Pada Windows, pengguna mengunduh *installer* dan mengikuti langkah pemasangan; pada macOS dan Linux tersedia paket melalui pengelola paket masing-masing. Setelah terpasang, pemasangan diperiksa dengan perintah berikut di Git Bash atau terminal.

```
git --version
```

Jika berhasil, terminal menampilkan nomor versi Git yang terpasang.

b. Konfigurasi awal

Git perlu mengetahui identitas pengguna karena setiap *commit* mencatat nama dan surel penulisnya. Konfigurasi ini cukup dilakukan sekali. Sebaiknya surel yang dipakai sama dengan surel akun GitHub.

```
git config --global user.name "M. Awaludin Ramadan"
git config --global user.email "email@contoh.com"
git config --global init.defaultBranch main
git config --list
```

Perintah pertama dan kedua mengatur nama dan surel, perintah ketiga mengatur nama cabang awal menjadi `main`, dan perintah terakhir menampilkan seluruh konfigurasi yang aktif.

c. Membuat repositori lokal

Sebagai contoh, penulis membuat folder baru bernama `catatan-git`, kemudian mengubahnya menjadi repositori Git.

```
mkdir catatan-git
cd catatan-git
git init
git status
```

`mkdir` membuat folder, `cd` berpindah ke dalam folder, `git init` membuat repositori (muncul folder tersembunyi `.git`), dan `git status` menampilkan keadaan repositori. Pada kondisi awal, `git status` menyatakan belum ada *commit* dan tidak ada berkas yang dilacak.

3.7 Cara Menghubungkan Git dengan GitHub

Repositori lokal dan repositori di GitHub adalah dua tempat yang terpisah. Agar keduanya terhubung, repositori lokal perlu diberi tahu alamat repositori di GitHub yang disebut *remote*. Nama bawaan untuk *remote* utama adalah `origin`.

a. Menambahkan remote

```
git remote add origin https://github.com/user/catatan-git.git
git remote -v
git branch -M main
```

Perintah pertama mendaftarkan alamat GitHub dengan nama `origin`. Perintah kedua menampilkan daftar *remote* untuk memastikan alamat sudah benar. Perintah ketiga mengganti nama cabang aktif menjadi `main` agar sama dengan GitHub.

b. Autentikasi

GitHub perlu memastikan bahwa orang yang mengirim perubahan adalah pemilik akun. Dokumentasi GitHub menyebutkan bahwa untuk operasi Git melalui HTTPS, kata sandi akun tidak lagi dapat dipakai dan digantikan oleh *personal access token* (PAT), sedangkan alternatifnya adalah kunci SSH (GitHub Docs, n.d.-b). Dua cara yang dipelajari adalah sebagai berikut.

- **HTTPS dengan token.** Token dibuat pada menu *Settings* akun GitHub bagian *Developer settings* lalu *Personal access tokens*. Ketika Git meminta kata sandi pada saat *push*, token dimasukkan sebagai pengganti kata sandi. Pada Windows, *Git Credential Manager* yang ikut terpasang dapat membantu menyimpan kredensial atau membuka *login* melalui peramban.
- **SSH.** Pengguna membuat sepasang kunci, yaitu kunci privat yang disimpan di komputer dan kunci publik yang didaftarkan di GitHub. Perintah dasarnya terlihat pada contoh di bawah.

```
ssh-keygen -t ed25519 -C "email@contoh.com"
cat ~/.ssh/id_ed25519.pub
ssh -T git@github.com
```

Perintah pertama membuat pasangan kunci, perintah kedua menampilkan kunci publik yang kemudian disalin dan ditempel pada menu *Settings*, bagian *SSH and GPG keys*, di GitHub. Perintah ketiga menguji koneksi. Bila memakai SSH, alamat *remote* berbentuk `git@github.com:namapengguna/catatan-git-github.git`. Kunci privat tidak boleh dibagikan kepada siapa pun.

3.8 Cara Melakukan Git Add, Commit, dan Push

Ketiga perintah ini merupakan alur yang paling sering dipakai: memilih perubahan, menyimpannya sebagai titik simpan, lalu mengirimkannya ke GitHub. Berikut contoh sederhana dengan membuat berkas `catatan.txt` di dalam folder repositori.

```
echo "Catatan hari pertama belajar Git" > catatan.txt
git status
git add catatan.txt
git status
git commit -m "Menambahkan catatan.txt"
git push -u origin main
```

Penjelasan langkah-langkah di atas adalah sebagai berikut.

1. `echo ... > catatan.txt` membuat berkas teks berisi satu kalimat.
2. `git status` yang pertama menampilkan `catatan.txt` sebagai berkas baru yang belum dilacak (*untracked*).
3. `git add catatan.txt` memasukkan berkas ke *staging area*. Untuk memasukkan semua perubahan sekaligus dapat dipakai `git add .`.
4. `git status` yang kedua menunjukkan berkas sudah siap di-*commit* (*changes to be committed*).
5. `git commit -m "..."` menyimpan isi *staging area* sebagai *commit* baru dengan pesan yang ditulis di antara tanda petik.
6. `git push -u origin main` mengirim *commit* ke cabang `main` pada *remote* `origin`. Opsi `-u` membuat Git mengingat tujuan ini sehingga selanjutnya cukup menulis `git push`.

Setelah perintah *push* berhasil, berkas dapat dilihat pada halaman repositori di GitHub setelah halaman dimuat ulang. Untuk melihat riwayat dapat dipakai perintah berikut.

```
git log --oneline
git diff
```

`git log --oneline` menampilkan riwayat *commit* dalam satu baris per *commit*, sedangkan `git diff` menampilkan perubahan yang belum di-*stage*.

Tips menulis pesan commit

- Tulis singkat, jelas, dan menjelaskan isi perubahan, misalnya `Menambah catatan perintah dasar Git`.

- Hindari pesan yang tidak informatif seperti `update` atau `coba`.
- Satu *commit* sebaiknya berisi satu kelompok perubahan yang saling berkaitan.

3.9 Cara Melakukan Git Pull dan Clone

a. Git clone

`git clone` dipakai untuk menyalin repositori yang sudah ada di GitHub ke komputer, lengkap dengan seluruh riwayatnya (GitHub Docs, n.d.-h). Alamat repositori diperoleh dari tombol hijau **Code** pada halaman repositori.

```
git clone https://github.com/namapengguna/catatan-git-
github.git
cd catatan-git-github
```

Perintah ini membuat folder baru bernama sama dengan repositori, sudah berisi berkas dan riwayat, serta langsung mengatur `origin` sebagai *remote*. Karena itu, *clone* tidak memerlukan `git init` dan `git remote add`.

b. Git pull

Misalkan repositori yang sama dikerjakan di dua tempat, misalnya di komputer rumah dan komputer kampus, atau oleh dua anggota kelompok. Apabila ada perubahan baru di GitHub, repositori lokal harus diperbarui dengan `git pull`.

```
git pull origin main
```

Perintah tersebut mengambil *commit* baru dari cabang `main` di `origin` kemudian menggabungkannya ke cabang lokal.

c. Git fetch dibandingkan git pull

`git pull` pada dasarnya adalah gabungan dari `git fetch` (mengambil data terbaru) dan penggabungan (*merge*). Bila pengguna ingin melihat dulu apa yang berubah sebelum menggabungkan, ia dapat memakai `git fetch origin` lalu memeriksanya. Kebiasaan yang baik adalah menjalankan `git pull` sebelum mulai bekerja dan sebelum melakukan `git push` agar tidak tertinggal dari versi terbaru.

3.10 Mengenal Branch dan Merge

Branch adalah jalur pengembangan yang terpisah. Dokumentasi GitHub menjelaskan bahwa percabangan memungkinkan pengguna bekerja pada fitur atau perbaikan tanpa memengaruhi versi utama (GitHub Docs, n.d.-i). Analoginya

seperti membuat salinan naskah untuk dicoret-coret; jika hasilnya bagus, perubahan dimasukkan ke naskah utama, jika tidak, salinan dibuang saja.

a. Perintah dasar branch

```
git branch
git branch fitur-judul
git switch fitur-judul
git switch -c fitur-lain
```

Perintah pertama menampilkan daftar cabang dan menandai cabang aktif. Perintah kedua membuat cabang `fitur-judul`. Perintah ketiga berpindah ke cabang tersebut. Perintah keempat membuat sekaligus berpindah ke cabang `fitur-lain`.

b. Merge

Setelah pekerjaan di cabang selesai dan sudah di-*commit*, perubahan digabungkan ke cabang utama dengan langkah berikut.

```
git switch main
git merge fitur-judul
git push origin main
```

Pertama pengguna berpindah ke cabang tujuan (`main`), kemudian menggabungkan cabang `fitur-judul` ke dalamnya, lalu mengirim hasilnya ke GitHub. Apabila cabang `main` tidak mengalami perubahan sejak cabang dibuat, Git cukup menggeser penunjuk ke depan yang disebut *fast-forward*. Apabila kedua cabang sama-sama berubah, Git membuat *merge commit* baru untuk menyatukan keduanya.

c. Konflik penggabungan

Konflik terjadi ketika dua cabang mengubah baris yang sama pada berkas yang sama sehingga Git tidak tahu versi mana yang harus dipakai. Git kemudian menghentikan penggabungan dan menandai berkas yang bermasalah dengan penanda seperti berikut.

```
<<<<<<< HEAD
Judul: Catatan Git
=======
Judul: Catatan Belajar Git dan GitHub
>>>>>>> fitur-judul
```

Cara menyelesaikannya adalah sebagai berikut.

1. Buka berkas yang berkonflik dan tentukan isi akhir yang diinginkan, apakah memakai salah satu versi atau menggabungkan keduanya.

2. Hapus seluruh penanda <<<<<<<, =======, dan >>>>>>>.
3. Jalankan `git add` pada berkas tersebut.
4. Jalankan `git commit` untuk menyelesaikan penggabungan.

3.11 Mengenal Fork dan Pull Request

Pada repositori milik orang lain, pengguna biasanya tidak memiliki izin untuk mengubah langsung. Untuk berkontribusi, pengguna memakai alur *fork* dan *pull request*. *Fork* adalah salinan repositori milik orang lain yang disimpan di akun pengguna sendiri, sehingga pengguna bebas mengubahnya tanpa memengaruhi repositori asli (GitHub Docs, n.d.-j). *Pull request* adalah permintaan kepada pemilik repositori asli agar perubahan yang telah dibuat ditinjau dan digabungkan (GitHub Docs, n.d.-k).

Alur kerja yang dipelajari adalah sebagai berikut.

1. Klik tombol **Fork** pada repositori yang ingin dikontribusi. Salinan repositori muncul di akun pengguna.
2. *Clone* hasil *fork* ke komputer dengan `git clone`.
3. Buat cabang baru, misalnya `git switch -c perbaiki-typo`.
4. Lakukan perubahan, lalu jalankan `git add` dan `git commit`.
5. Kirim cabang ke GitHub dengan `git push origin perbaiki-typo`.
6. Pada GitHub, klik **Compare & pull request**, isi judul dan penjelasan perubahan, lalu klik **Create pull request**.
7. Pemilik repositori meninjau perubahan, dapat memberi komentar atau meminta perbaikan, kemudian menekan **Merge pull request** bila setuju.

Agar *fork* tetap sama dengan repositori asli, pengguna dapat menambahkan repositori asli sebagai *remote* tambahan bernama `upstream`.

```
git remote add upstream https://github.com/pemilik/proyek.git
git fetch upstream
git merge upstream/main
```

Pull request juga digunakan di dalam satu repositori yang sama oleh tim, yaitu untuk menggabungkan cabang fitur ke cabang `main` setelah ditinjau anggota lain. Dengan cara ini kualitas perubahan lebih terjaga. Fitur *Issues* dapat dipakai bersamaan, misalnya dengan menulis `Closes #12` pada deskripsi *pull request* agar *issue* nomor 12 otomatis tertutup setelah penggabungan.

3.12 Keamanan dan Pengelolaan Repository

a. Keamanan akun dan data

- **Autentikasi dua faktor (2FA).** Menambah lapisan keamanan karena selain kata sandi diperlukan kode tambahan dari aplikasi autentikator atau perangkat lain.
- **Kata sandi kuat dan unik.** Jangan memakai kata sandi yang sama dengan layanan lain.
- **Menjaga token dan kunci SSH.** Token dan kunci privat diperlakukan seperti kata sandi: tidak dibagikan, tidak ditulis di dalam kode, dan segera dicabut (*revoke*) apabila bocor.
- **Tidak mengunggah data rahasia.** Berkas seperti `.env`, kunci API, dan kata sandi basis data jangan di-*commit*. Daftarkan berkas tersebut pada `.gitignore`.
- **Menangani kebocoran.** Jika rahasia terlanjur terunggah, ganti atau cabut rahasia itu terlebih dahulu. Menghapus berkas pada *commit* berikutnya tidak cukup karena riwayat lama masih menyimpannya.

Contoh isi berkas `.gitignore`:

```
# berkas rahasia
.env
# berkas sementara
*.log
__pycache__/
# folder hasil pengolahan
output/
```

b. Pengelolaan repositori

Pengaturan repositori berada pada tab **Settings**. Beberapa hal yang dipelajari adalah sebagai berikut.

- **Visibilitas.** Repositori dapat diubah antara publik dan privat sesuai kebutuhan.
- **Kolaborator.** Pemilik dapat mengundang orang lain dan mengatur tingkat izin, misalnya hanya membaca (*read*) atau dapat menulis (*write*).
- **Perlindungan cabang (*branch protection*).** Cabang `main` dapat diatur agar perubahan hanya masuk melalui *pull request* yang sudah ditinjau.
- **Issues, Labels, dan Milestones.** Dipakai untuk mencatat dan mengelompokkan pekerjaan.

- **Tag dan Release.** Menandai versi tertentu proyek, misalnya `v1.0`.
- **Lisensi.** Menentukan hak orang lain untuk memakai kode. Repositori publik tanpa lisensi tetap dilindungi hak cipta sehingga orang lain belum tentu boleh menggunakannya.
- **Peringatan keamanan.** GitHub menyediakan fitur seperti pemindaian rahasia dan Dependabot yang memberi peringatan terhadap kerentanan pada pustaka yang digunakan.
- **Arsip dan hapus.** Repositori yang sudah tidak aktif dapat diarsipkan agar menjadi hanya-baca, atau dihapus pada bagian *Danger Zone* setelah melalui konfirmasi.

c. Membatalkan kesalahan

Kesalahan dapat terjadi, sehingga perlu diketahui cara memperbaikinya dengan aman. Pada berkas yang belum di-*commit*, `git restore namaberkas` mengembalikan berkas ke isi *commit* terakhir, sedangkan `git restore --staged namaberkas` mengeluarkan berkas dari *staging area*. Pada *commit* yang sudah dikirim ke GitHub, cara yang lebih aman adalah `git revert`, karena membuat *commit* baru yang membatalkan perubahan tanpa menghapus riwayat. Perintah `git reset --hard` bersifat berbahaya karena dapat menghapus perubahan secara permanen, sehingga harus dipakai dengan hati-hati.

BAB IV
HASIL PEMBELAJARAN

4.1 Pemahaman Mengenai Git dan GitHub

Dari kegiatan praktikum ini penulis memahami bahwa Git dan GitHub adalah dua hal yang saling melengkapi tetapi tidak sama. Git adalah alat yang bekerja di komputer untuk mencatat riwayat perubahan berkas, sedangkan GitHub adalah layanan di internet untuk menyimpan repositori Git dan bekerja sama dengan orang lain. Sebelumnya penulis mengira keduanya adalah satu aplikasi yang sama. Setelah mempelajari bahan resmi, penulis menyadari bahwa Git dapat dipakai sendiri tanpa GitHub, sementara GitHub hanya bermanfaat jika ada repositori Git di dalamnya.

Penulis juga memahami bahwa kunci dari sistem kontrol versi adalah kebiasaan mencatat perubahan secara bertahap dengan pesan yang jelas. Dengan kebiasaan ini, penulis tidak perlu lagi membuat banyak salinan berkas dengan nama yang membingungkan.

4.2 Pemahaman Fungsi Repository

Penulis memahami bahwa repositori bukan sekadar folder penyimpanan. Repositori adalah folder proyek yang menyimpan berkas sekaligus seluruh riwayat perubahannya melalui folder tersembunyi `.git`. Terdapat repositori lokal di komputer dan repositori *remote* di GitHub, dan keduanya perlu disinkronkan dengan perintah `git push` dan `git pull`. Penulis juga mengetahui bahwa pemilihan visibilitas publik atau privat perlu dilakukan dengan cermat, terutama bila repositori berisi tugas yang belum boleh dibagikan atau data yang bersifat pribadi.

Selain itu, penulis memahami bahwa kualitas sebuah repositori dipengaruhi oleh kelengkapannya, antara lain nama yang jelas, README.md yang informatif, `.gitignore` yang sesuai, serta lisensi bila repositori ingin dibagikan.

4.3 Pemahaman Perintah Dasar Git

Penulis memahami bahwa perintah Git tidak harus dihafal sekaligus. Perintah yang paling sering digunakan dapat dikelompokkan menurut tahapan kerja seperti pada Tabel 4.1.

Tabel 4.1 Pengelompokan Perintah Git Menurut Tahapan Kerja

Tahap	Perintah	Tujuan
Persiapan	`git config`, `git init`, `git clone`	Mengatur identitas dan memperoleh repositori
Pencatatan	`git status`, `git add`, `git commit`	Memeriksa, memilih, dan menyimpan perubahan
Pemeriksaan	`git log`, `git diff`	Melihat riwayat dan perbedaan
Percabangan	`git branch`, `git switch`, `git merge`	Bekerja di jalur terpisah lalu menggabungkan
Sinkronisasi	`git remote`, `git push`, `git pull`, `git fetch`	Menghubungkan dan menyamakan dengan GitHub
Perbaikan	`git restore`, `git revert`	Membatalkan kesalahan dengan aman

Urutan yang paling penting dikuasai penulis adalah `git status`, `git add`, `git commit`, dan `git push`. Penulis juga belajar bahwa `git status` sebaiknya sering dijalankan karena menunjukkan keadaan terkini dan petunjuk langkah selanjutnya.

4.4 Pemahaman Proses Upload File ke GitHub

Penulis memahami dua cara mengunggah berkas. Cara pertama melalui peramban dengan menu **Add file** lalu **Upload files**, yang mudah dan cocok untuk berkas sedikit. Cara kedua melalui Git di komputer dengan urutan `git add`, `git commit`, dan `git push`, yang lebih sesuai untuk pekerjaan berkelanjutan karena riwayat tercatat rapi dan dapat dilakukan berulang dengan cepat.

Penulis juga mempelajari beberapa hal yang perlu diperhatikan sebelum mengunggah, yaitu memastikan alamat *remote* benar, melakukan autentikasi dengan token atau SSH, memastikan tidak ada berkas rahasia yang ikut terunggah, serta menulis pesan *commit* yang bermakna.

4.5 Manfaat GitHub dalam Dunia Pemrograman

Dalam dunia pemrograman, penulis menyimpulkan bahwa GitHub memberikan manfaat berikut.

- **Kolaborasi tim.** Beberapa pengembang dapat bekerja pada proyek yang sama dengan alur *branch* dan *pull request*.

- **Peninjauan kode.** Perubahan dapat ditinjau terlebih dahulu sebelum masuk ke versi utama sehingga kesalahan lebih cepat ditemukan.
- **Pelacakan pekerjaan.** *Issues* dan *Projects* membantu membagi dan memantau tugas.
- **Dokumentasi dan transparansi.** README.md, wiki, dan riwayat *commit* membuat proyek mudah dipahami orang lain.
- **Ekosistem *open source*.** Pengembang dapat mempelajari, memakai, dan berkontribusi pada proyek orang lain.
- **Otomatisasi.** *GitHub Actions* mengurangi pekerjaan berulang, misalnya menjalankan pengujian otomatis.
- **Portofolio.** Repositori yang rapi dapat menjadi bukti kemampuan ketika melamar magang atau pekerjaan.

4.6 Penerapan GitHub dalam Kegiatan Perkuliahan

Menurut penulis, GitHub dapat diterapkan dalam kegiatan perkuliahan dengan cara berikut.

- **Menyimpan catatan dan tugas.** Satu repositori dapat dipakai untuk mengumpulkan catatan, kode latihan, dan laporan setiap mata kuliah, sehingga tersusun rapi dan tidak hilang ketika perangkat bermasalah.
- **Tugas kelompok.** Anggota kelompok dapat bekerja pada cabang masing-masing, kemudian digabungkan melalui *pull request* sehingga kontribusi setiap anggota tercatat.
- **Pengumpulan tugas.** Dosen dapat meminta mahasiswa mengumpulkan tautan repositori, dan riwayat *commit* dapat menjadi bukti proses pengerjaan.
- **Belajar dari proyek lain.** Mahasiswa dapat membaca kode dan dokumentasi proyek *open source* untuk memperluas wawasan.
- **Dukungan berbagai mata kuliah.** Dalam mata kuliah seperti Pengolahan Citra Digital, GitHub dapat membantu merapikan versi kode, catatan percobaan, dan dokumentasi, tanpa perlu mengubah cara kerja inti mata kuliah tersebut.
- **Membangun portofolio.** Repositori yang dikelola baik sejak semester awal dapat menjadi rekam jejak pembelajaran.

BAB V
PENUTUP

5.1 Kesimpulan

Berdasarkan uraian pada bab-bab sebelumnya, dapat disimpulkan hal-hal berikut.

1. Git adalah sistem kontrol versi terdistribusi yang bekerja di komputer untuk mencatat dan mengelola riwayat perubahan berkas, sedangkan GitHub adalah platform *hosting* repositori Git berbasis web yang menambahkan fitur kolaborasi.
2. Git dibuat oleh Linus Torvalds pada tahun 2005, sedangkan GitHub didirikan pada tahun 2008 dan sejak tahun 2018 menjadi bagian dari Microsoft.
3. Repositori adalah tempat penyimpanan proyek beserta seluruh riwayatnya, dan dapat berada di komputer lokal maupun di GitHub.
4. Perintah dasar Git yang perlu dikuasai adalah `git init`, `git clone`, `git status`, `git add`, `git commit`, `git push`, dan `git pull`, dilengkapi `git branch`, `git switch`, dan `git merge` untuk percabangan.
5. *Fork* dan *pull request* memungkinkan kontribusi pada repositori milik orang lain secara tertata, sedangkan fitur keamanan seperti 2FA, token, SSH, dan `.gitignore` penting untuk menjaga akun dan data.
6. Git dan GitHub bermanfaat bagi mahasiswa dalam mengelola tugas, bekerja dalam kelompok, dan membangun portofolio.

5.2 Saran

Penulis menyampaikan beberapa saran sebagai berikut.

- Mahasiswa disarankan membiasakan Git dan GitHub sejak awal untuk setiap tugas, tidak hanya saat praktikum.
- Tulis pesan *commit* yang jelas dan periksa isi repositori sebelum *push* agar data rahasia tidak ikut terunggah.
- Pembelajaran lanjutan dapat mencakup penyelesaian konflik pada kerja kelompok, *GitHub Actions*, serta *Issues* dan *Projects*.
- Rujuk dokumentasi resmi karena tampilan dan fitur GitHub dapat berubah dari waktu ke waktu.

DAFTAR PUSTAKA

Chacon, S., & Straub, B. (2014). *Pro Git* (2nd ed.). Apress. https://git-scm.com/book/en/v2

Git. (n.d.-a). *Git: Fast version control* [Situs resmi]. https://git-scm.com/

Git. (n.d.-b). *A short history of Git*. Dalam *Pro Git*. https://git-scm.com/book/en/v2/Getting-Started-A-Short-History-of-Git

Git. (n.d.-c). *About version control*. Dalam *Pro Git*. https://git-scm.com/book/en/v2/Getting-Started-About-Version-Control

Git. (n.d.-d). *Downloads*. https://git-scm.com/downloads

GitHub Docs. (n.d.-a). *About GitHub and Git*. https://docs.github.com/en/get-started/start-your-journey/about-github-and-git

GitHub Docs. (n.d.-b). *About authentication to GitHub*. https://docs.github.com/en/authentication/keeping-your-account-and-data-secure/about-authentication-to-github

GitHub Docs. (n.d.-c). *About repositories*. https://docs.github.com/en/repositories/creating-and-managing-repositories/about-repositories

GitHub Docs. (n.d.-d). *Create a repo*. https://docs.github.com/en/repositories/creating-and-managing-repositories/quickstart-for-repositories

GitHub Docs. (n.d.-e). *Adding a file to a repository*. https://docs.github.com/en/repositories/working-with-files/managing-files/adding-a-file-to-a-repository

GitHub Docs. (n.d.-f). *About READMEs*. https://docs.github.com/en/repositories/managing-your-repositorys-settings-and-features/customizing-your-repository/about-readmes

GitHub Docs. (n.d.-g). *Basic writing and formatting syntax*. https://docs.github.com/en/get-started/writing-on-github/getting-started-with-writing-and-formatting-on-github/basic-writing-and-formatting-syntax

GitHub Docs. (n.d.-h). *Cloning a repository*. https://docs.github.com/en/repositories/creating-and-managing-repositories/cloning-a-repository

GitHub Docs. (n.d.-i). *About branches*. https://docs.github.com/en/pull-requests/collaborating-with-pull-requests/proposing-changes-to-your-work-with-pull-requests/about-branches

GitHub Docs. (n.d.-j). *About forks*. https://docs.github.com/en/pull-requests/collaborating-with-pull-requests/working-with-forks/about-forks

GitHub Docs. (n.d.-k). *About pull requests*. https://docs.github.com/en/pull-requests/collaborating-with-pull-requests/proposing-changes-to-your-work-with-pull-requests/about-pull-requests

Microsoft. (2018a, 4 Juni). *Microsoft to acquire GitHub for $7.5 billion*. Microsoft News Center. https://news.microsoft.com/2018/06/04/microsoft-to-acquire-github-for-7-5-billion/

Microsoft. (2018b, 26 Oktober). *Microsoft completes GitHub acquisition*. Microsoft News Center. https://news.microsoft.com/2018/10/26/microsoft-completes-github-acquisition/